

Universidad de Costa Rica

Escuela de Ciencias de la Computación e Informática

Ingeniería de Computación con Varios Énfasis

Análisis de Algoritmos y Estructuras de Datos



Tarea III

Reporte: Análisis de Rutas Óptimas entre Ciudades usando Grafos Dirigidos y Floyd-Warshall.

Estudiante:

Alexa Alpízar Mora, C20281

E-mail:

alexa.alpizar@ucr.ac.cr

Docente:

Prof. Dr Allan Berrocal Rojas

CI-0116

Ciclo I, 2025

I. Nombre: Alexa Alpízar Mora, carné C20281.

II. Planeamiento del problema a resolver.

El reporte titulado personalmente como “Análisis de Rutas Óptimas entre Ciudades usando Grafos Dirigidos y Floyd-Warshall.”, tiene como objetivo principal utilizar teoría de grafos y algoritmos asociados aplicado a la resolución de un problema específico. Se deberá analizar el problema indicado por el profesor y diseñar una solución utilizando el conocimiento adquirido a partir del estudio de los algoritmos de grafos vistos en clase. Específicamente, se proporcionan tres archivos tipo .csv con listas de distintos tamaños; “*small*”, “*medium*” y “*large*” que contienen la información de ciudades que están conectadas directamente por un camino junto con el tiempo de duración que se tardaría yendo de una a la otra.

Algunas ciudades no están conectadas directamente, pero se puede llegar a ellas por una o más ciudades intermediarias en el trayecto. De esta manera se creará un programa que lea el archivo seleccionado con la información de la simulación del “SiNTiEmPO” que la procese y cree un grafo dirigido $G = (V, E)$, donde los vértices/nodos representen las ciudades donde se ubica un centro SiNTiEmPo y las aristas que los conectan tienen un peso w que representa la duración de viaje entre cada par de los centros del sistema (conectados directamente). Luego, el programa extraerá información del grafo para responder ciertas preguntas solicitadas. Por último, debe haber un mecanismo que permita al usuario consultar estas preguntas frecuentes y conocer sus respuestas. Cabe recalcar que las preguntas a resolver son las siguientes:

1. Encontrar el nombre de la ciudad donde es más efectivo colocar mayor capacidad de equipo de manera que si se necesita repartir equipo ubicado en este centro hacia todos los demás, se tenga que invertir la menor cantidad de tiempo total de viajes. Muestre el nombre de la ciudad y el valor de tiempo correspondiente. Si hay más de una ciudad que reúne esta condición, liste todos los nombres.

2. Dada una cantidad x , encuentre la ciudad y desde la cual es mejor despachar equipo de apoyo o suministros hacia la ciudad x para que llegue lo más pronto posible en caso de que ocurra una emergencia en la ciudad x . Muestre el nombre de la ciudad y así como el valor del tiempo y la ruta que se debe seguir para llegar a la ciudad y a la x . Si hay más de una ciudad que reúne esta condición, liste todos los nombres.

3. Encontrar el par de ciudades (x,y) que están más distantes la una de la otra e indicar el tiempo de viaje entre ambas así como la ruta que se debe seguir para llegar de la ciudad x a la y . Si hay más de una ciudad que reúne esta condición, liste todos los nombres.

4. Encontrar el par de ciudades (x,y) que están menos distantes la una de la otra e indicar el tiempo de viaje entre ambas así como la ruta que se debe seguir para llegar de la ciudad x a la y . Si hay más de una ciudad que reúne esta condición, liste todos los nombres.

5. Producir una lista de todas las ciudades en orden creciente según el tiempo de viaje promedio desde esa ciudades hasta todas las demás ciudades. Muestre el nombre de la ciudad así como el tiempo promedio de viaje de dicha ciudad hasta todas las demás.

III. Descripción de la solución.

Primeramente, para la solución se decidió llevarla a cabo alrededor de la búsqueda de respuestas para las preguntas proporcionadas. Es decir, de acuerdo a las respuestas que se necesitaban así sería como elegiría el algoritmo adecuado para utilizar en el grafo y extraer la información. Cada consulta corresponde a

una función que hace uso el algoritmo elegido. En este caso se eligió Floyd- Warshal debido a que permite calcular todas las distancias mínimas en grafos dirigidos y ponderados, aún cuando algunas ciudades no están conectadas directamente (Cormen et al., 2009). Debido a que el grafo realizado con la información del archivo proporcionado cumple con estas características, se decidió elegir el algoritmo mencionado. Además según lo estudiado en clase, Floyd-Warshall tiene una duración de $O(V^3)$ (cita), lo cual lo hace apropiado para grafos densos. Consiste en inicializar una matriz de distancias con infinito (excepto la diagonal principal, que se establece en cero, indicando que la distancia de una ciudad a sí misma es cero) y luego, a partir de la lista de adyacencia del grafo, se colocan las distancias directas entre ciudades conectadas y se registra el nodo siguiente en el camino. Mediante tres bucles, el algoritmo verifica si pasando por un nodo intermedio se puede encontrar un camino más corto entre dos ciudades. Si es así, se actualiza la distancia y también el camino registrado. Al finalizar, se obtiene una matriz que contiene la menor distancia entre cada par de ciudades y otra que permite reconstruir las rutas óptimas correspondientes.

Luego, se continuó con la “base” del código según se indicaba en el enunciado, debido que independientemente del algoritmo elegido, debía cumplirse con las funciones de leer un archivo, procesar la información y generar un grafo. Es por esta razón, que de manera modular se pensaron dos clases: File_Reader y Graph. La primera se encarga únicamente de leer el archivo proporcionado (.csv) y se separa por comas la información dada para luego extraer lo relevante; los nombres de las ciudades “nameSrc” y “nameTgt” además del peso de las aristas “weightStr”. También, su método readCSV() devuelve un vector de registros (tuplas) con la ciudad de origen, ciudad de destino y el peso (tiempo) que se utilizarán para después construir el grafo.

Seguidamente la clase Graph implementa la estructura de un grafo que utiliza un mapa de listas de adyacencias para representar las conexiones entre ciudades, una lista de vértices/nodos y un mapa de índices para traducir los nombres de ciudades a índices numéricos para trabajar con matrices. Mediante el método buildFromEdges (), se construye el grafo y se consulta su estructura mediante getters. De ser necesario también se puede imprimir la lista de adyacencia con los propósitos de visualización y depuración.

Por otra parte, de acuerdo a la solución de las preguntas a responder posiblemente consultadas por el usuario, se creó la clase Graph_Analyzer, en la cual se realiza el análisis del grafo que con anterioridad se construye a partir de las demás clases. En Graph_Analyzer principalmente; se implementa el algoritmo de Floyd-Warshall, se reconstruyen rutas mínimas y se implementan los métodos para encontrar la ciudad central(más cercana a las demás), la mejor para despachar suministros a otra, las más cercanas y lejanas (una de otra) y por último el ranking de ciudades por tiempo promedio de forma creciente. Específicamente las funciones de estos métodos son:

floydWarshal(): Realiza el algoritmo, llena una matriz con las distancias mínimas entre todos los pares de vértices(nodos) y una matriz de predecesores para reconstruir caminos. De esta manera se conocen las distancias mínimas entre ciudades que posteriormente se utilizarán.

reconstructpPath(): Reconstruye el camino mínimo entre los nodos u y v usando la matriz de predecesores para conocer el camino específico que debe seguirse, lo cual es necesario para el análisis de rutas óptimas.

findCenter(): Con la matriz de distancias encuentra la ciudad más céntrica, cuya suma de distancias a todas las demás es la menor y responde a la primera pregunta.

findBestDispatchCity(): Recibe una ciudad destino y encuentra la otra que puede despachar suministros hacia ella con el menor tiempo de viaje posible, utilizando el algoritmo elegido, para responder la segunda pregunta. También, imprime la ciudad y la ruta hacia su destino.

findMostDistantCities(): Encuentra el par de ciudades más alejadas entre sí y genera un archivo con la información para responder la pregunta tres.

findClosestCities(): De igual manera que en findMostDistantCities(), sin embargo encontrando las ciudades más cercanas entre sí y esta vez responder la cuarta pregunta.

rankCitiesByAverageDistance(): Se encarga de calcular y ordenar de manera creciente las ciudades por su tiempo de duración hacia las demás (según el algoritmo de Floyd-Warshall) y poder llevar a cabo el ranking de la solicitud 5.

Para concluir, se implementó la clase Menu que funciona como interfaz en la consola de la terminal y gestiona la interacción con el usuario por medio de un menú de opciones en la cual el usuario puede elegir alguna de las preguntas para obtener una respuesta. Se separó del main para continuar con la estructura modular y facilitar pruebas o cambios.

IV. Resultados

Los resultados obtenidos en las pruebas realizadas con los archivos “input_large.csv”, input_medium.csv” e “input_small.csv” se pueden observar en la carpeta denominada “doc” de la entrega de la Tarea Programada 3 (tp3) por medio del repositorio del curso de Análisis de Algoritmos ci0116_25a_c20281 de GitHub.

V. Documentación

Para acceder a la documentación sobre cómo correr e interactuar con el programa, puede dirigirse al rearme.md de la entrega de la Tarea Programada 3 (tp3) por medio del repositorio del curso de Análisis de Algoritmos ci0116_25a_c20281 de GitHub.